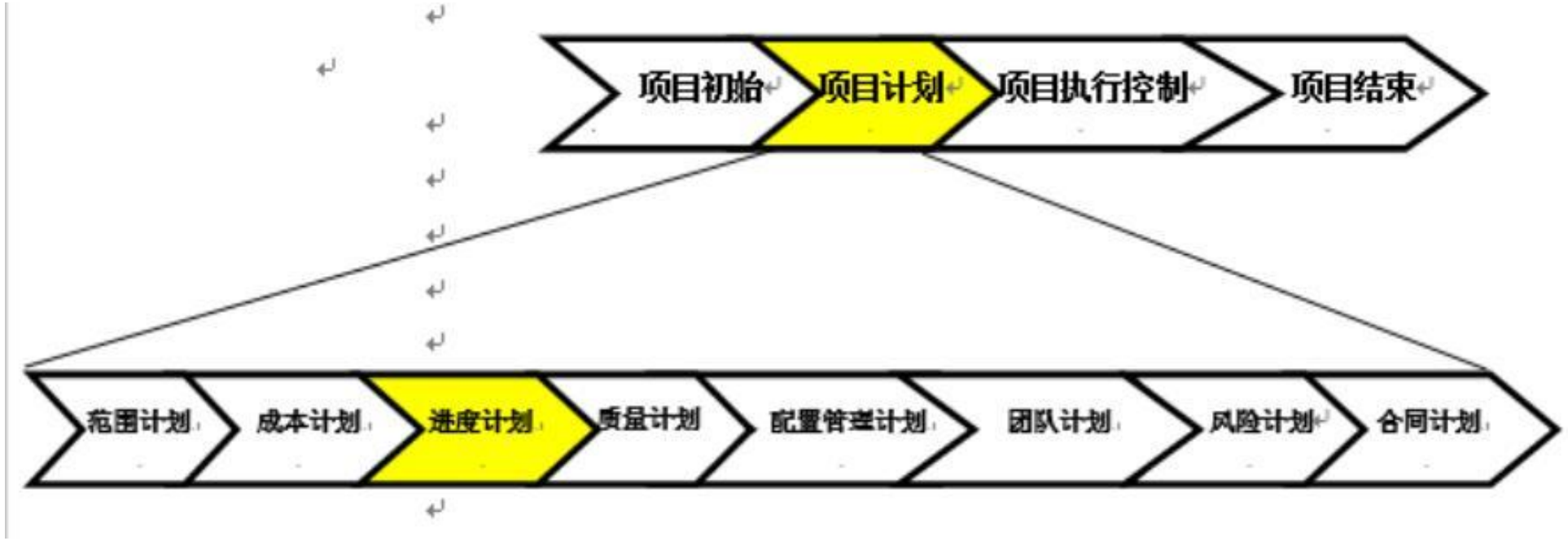
IT项目管理

史建锋

2025年12月10日

进 度 计 划


项目初始
项目计划
项目执行控制
项目结束
范围计划
成本计划
进度计划
质量计划
配置管理计划
团队计划
风险计划
合同计划

第7章

IT项目进度管理

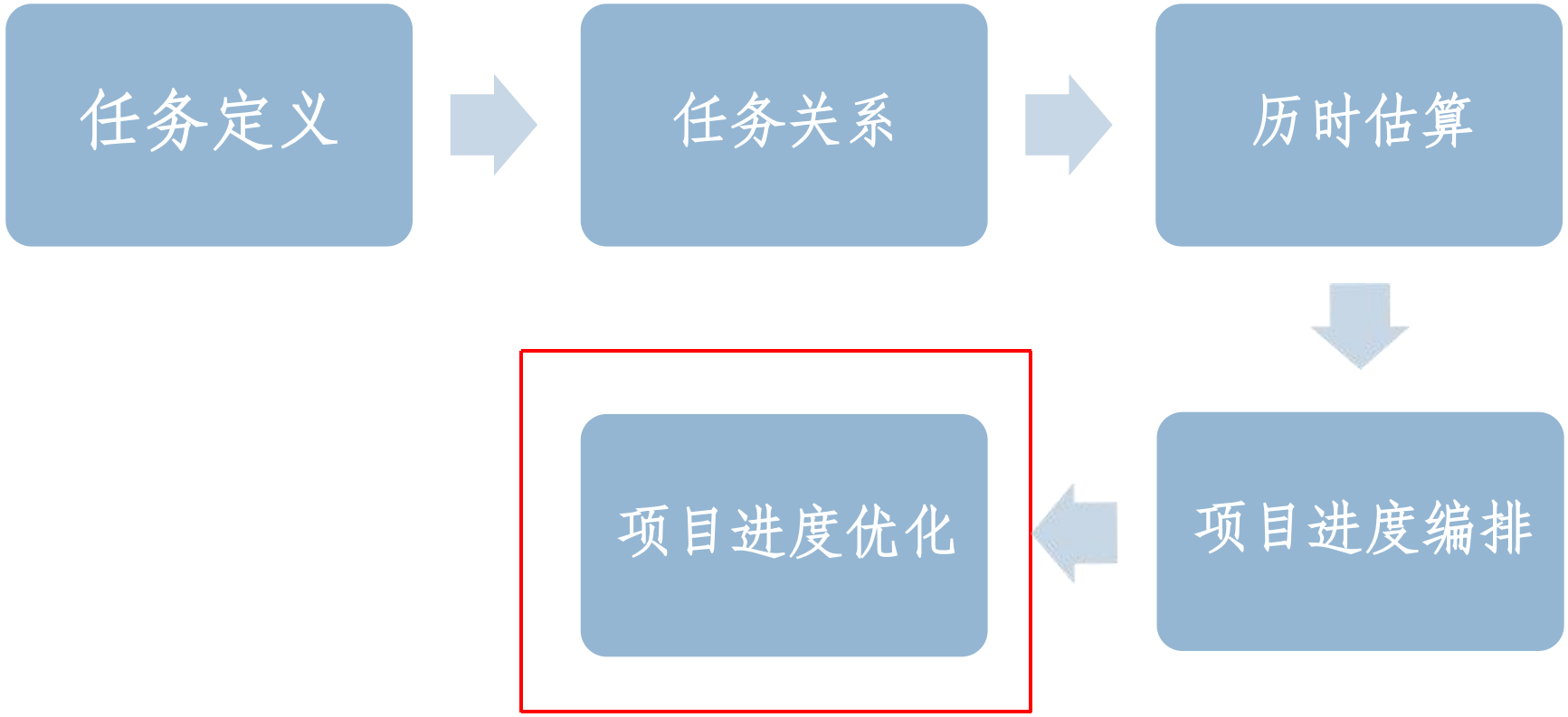
本 章 要 点

一 进度计划及其重要性

二 传统项目进度计划

三 敏捷项目进度计划

项目进度计划过程



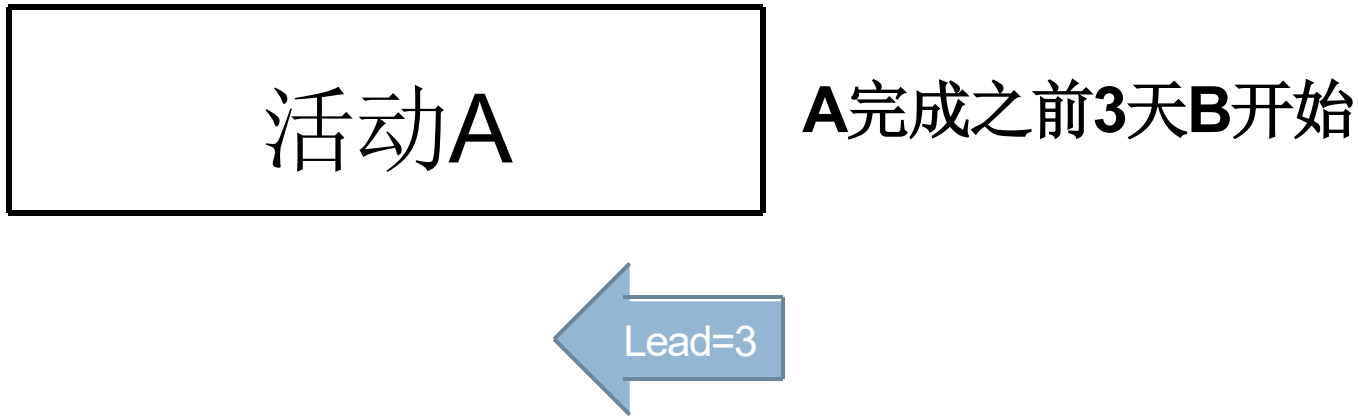
超前(Lead)与滞后(Lag)

- 时间压缩法
- 资源优化

任务滞后Lag举例



任务超前(Lead)



作用：

1）解决任务的搭接

2）对任务可以进行合理的拆分

3）缩短项目工期

- 超前(Lead)与滞后(Lag)
- 时间压缩法
- 资源优化

时间压缩法

时间压缩法是在不改变项目范围的前提下缩短项目工期的方法

- 应急法--赶工 (Crash)
- 平行作业法--快速跟进

应急法——赶工

1. 进度压缩单位成本方法
   - 可压缩范围，线性关系
2. Charles Symons(1991)方法
   - 非线性关系，有压缩限制

线性关系：进度压缩单位成本方法

压缩范围：正常值与可压缩值之间

例如：

- 任务A：正常进度7周，成本50万；压缩到5周的成本是62万
- 压缩范围：7周——5周

线性关系：进度压缩单位成本方法

压缩范围：正常值与可压缩值之间

- 项目活动的正常值
  - 正常历时
  - 正常成本
- 项目活动的压缩值
  - 压缩历时
  - 压缩成本

线性关系：进度压缩单位成本方法

进度压缩单位成本=（压缩成本-正常成本）/(正常进度-压缩进度)

例:

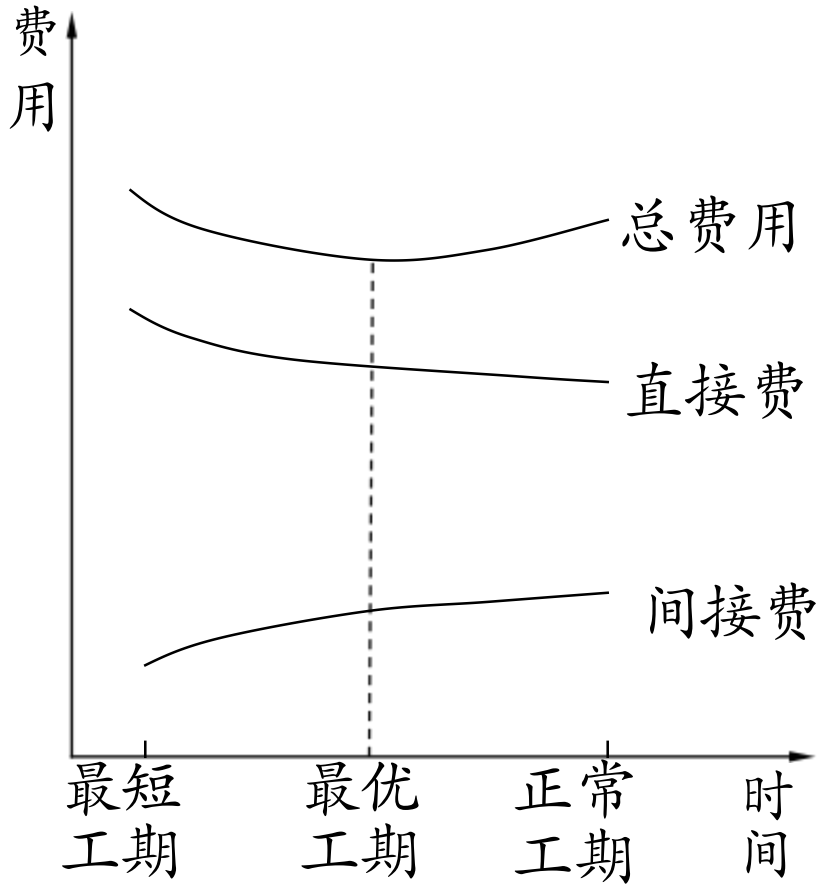
- 任务A：正常进度7周，成本50万；压缩到5周的成本是62万
- 进度压缩单位成本=(62-50)/(7-5)=6万元/周
- 如果压缩到6周的成本是：56万

- **非线性进度压缩法**

(1)**直接费用**：完成各项工作或工序直接所需人力、资源、设备等费用。如：设备、能源及材料消耗等直接与完成工序有关的费用，它随工期的缩短而增加。

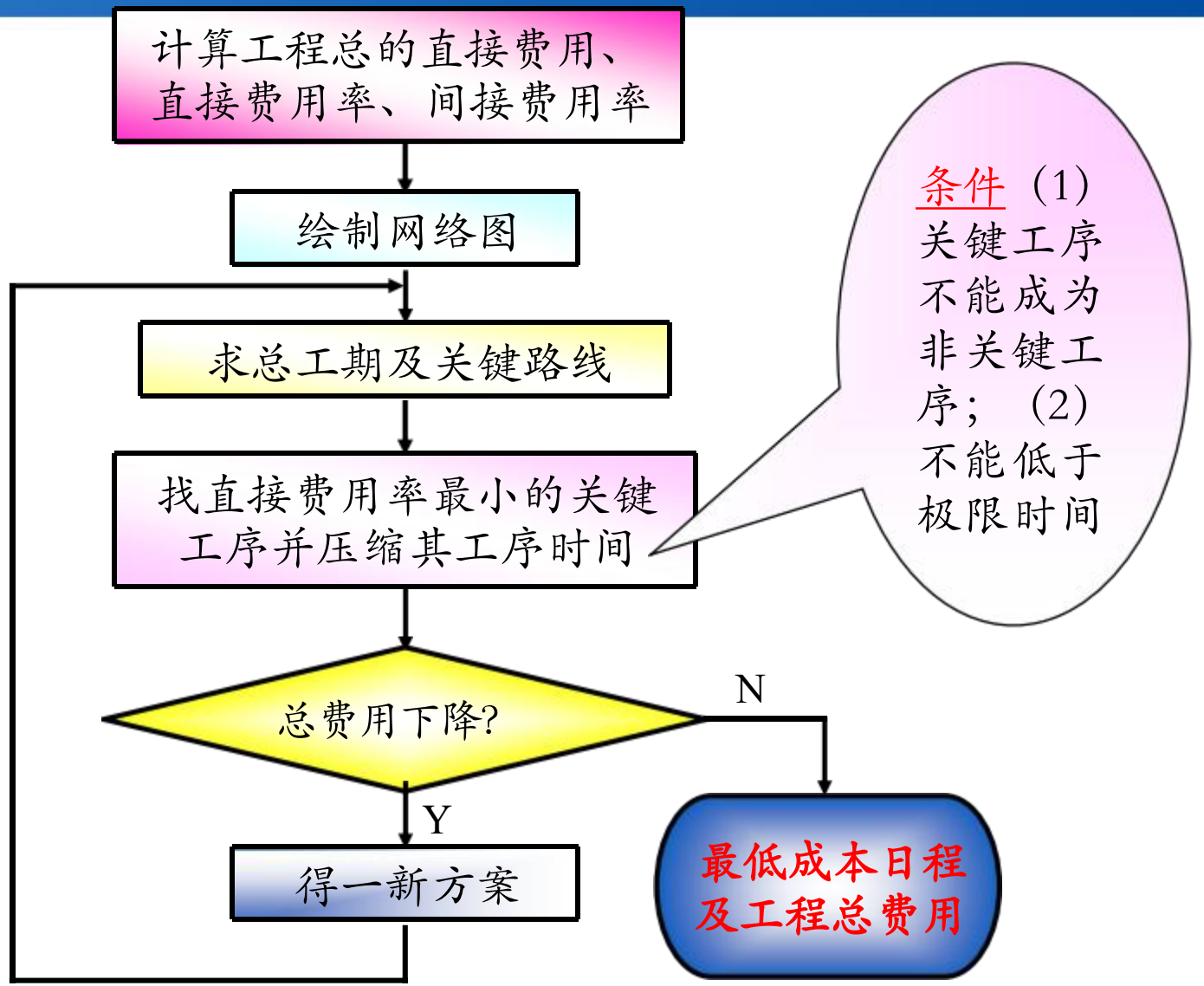
(2)**间接费用**：办公费、管理费等，它随工期的缩短而减少。

直接费、间接费、总费用与工程完工期的关系如图：

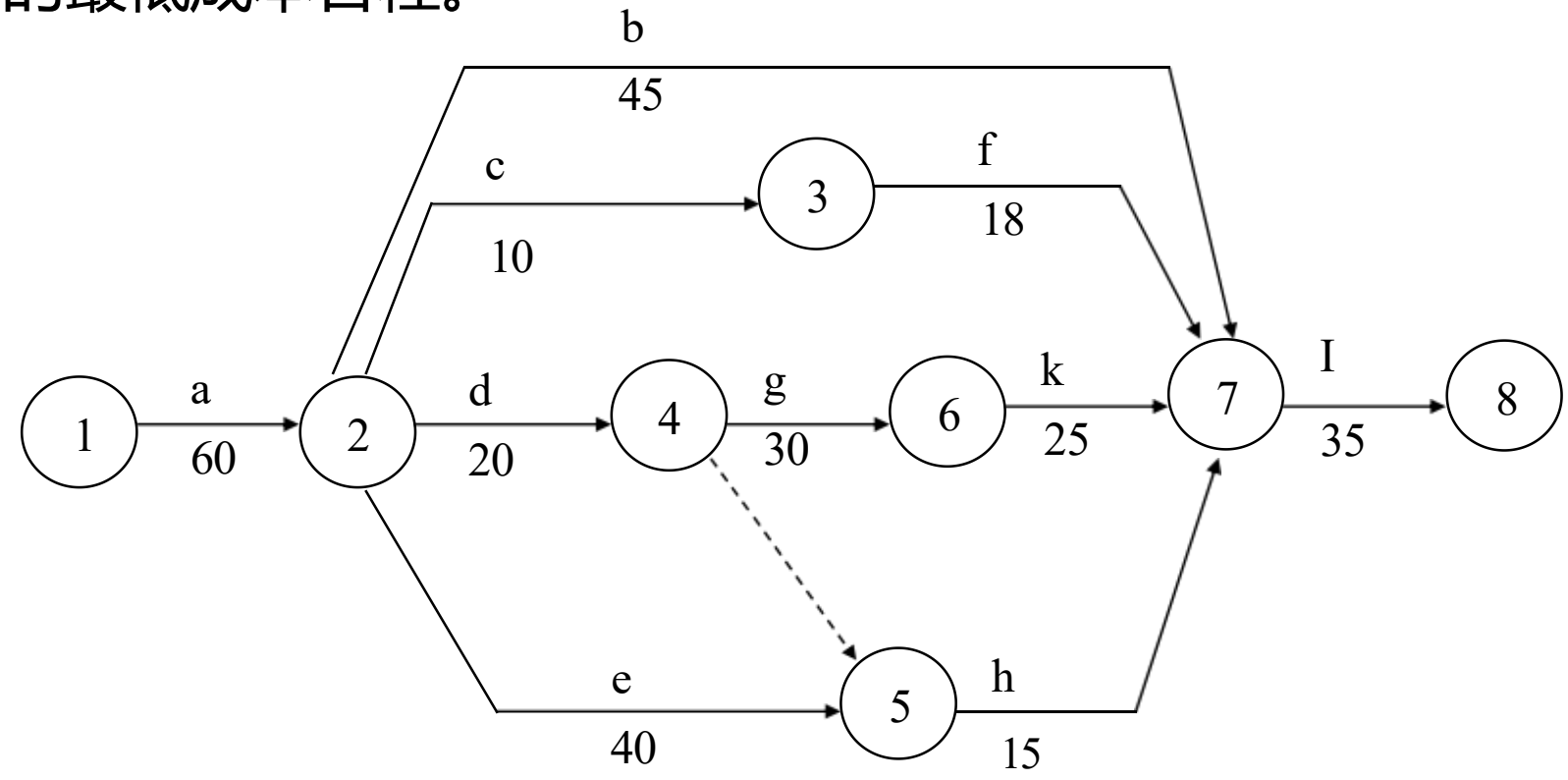


工期－费用曲线

$$费用率=\frac{最短工期费用-正常工期费用}{正常工期-最短工期}$$

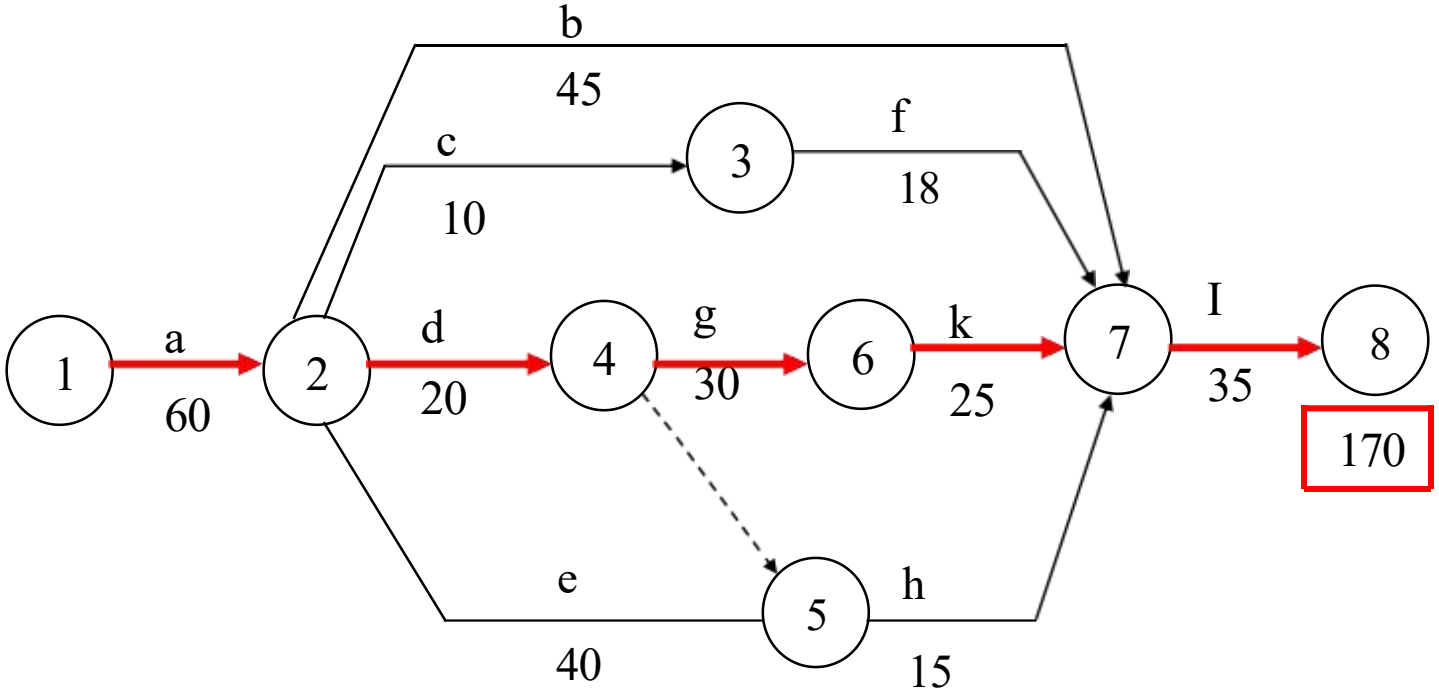


例：某项目网络图如下，已知该项目各道工序的正常作业时间、最短作业时间，及其直接费用（见下页表）；又已知此项目每天的间接费用为400元，求该项目的最低成本日程。



工序	正常情况下（天）		采取措施后（天）		直接费用率（元/天）	间接费用率（元/天）
	工序时间	直接费用	最快时间	直接费用		
a	60	10000	60	10000	- -	400
b	45	4500	30	6300	120	
c	10	2800	5	4300	300	
d	20	7000	10	11000	400	
e	40	10000	35	12500	500	
f	18	3600	10	5440	230	
g	30	9000	20	12500	350	
h	15	3750	10	5750	400	
k	25	6250	15	9150	290	
I	35	12000	35	12000	- -	

首先，先寻找关键路径

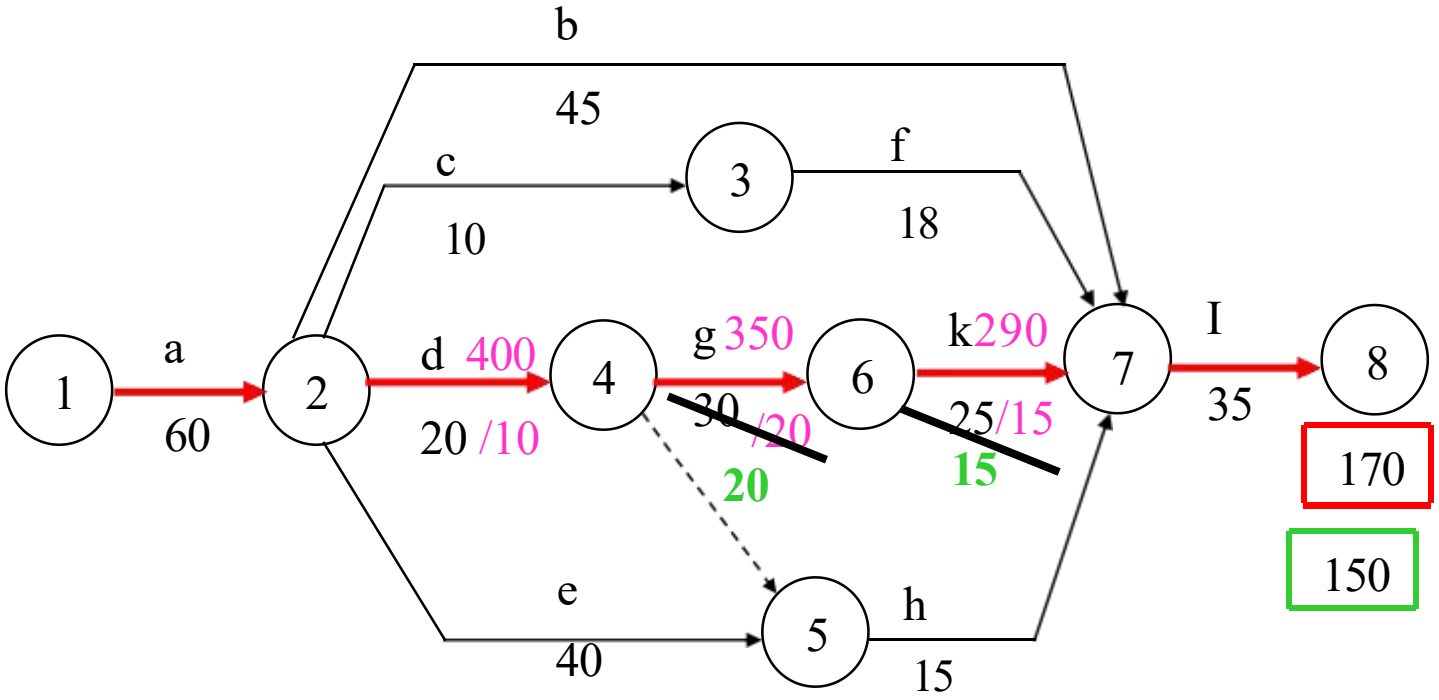


第二步，求解原工期及费用。通过关键路径可知，项目的完工期为170天。

- ✓ 直接费用为： 68900 (元)
- ✓ 间接费用为：170×400＝68000 (元)
- ✓ 总费用为： 68900＋68000＝136900 (元)

——方案Ⅰ

第三步，尝试压缩关键路径上的工序时间。

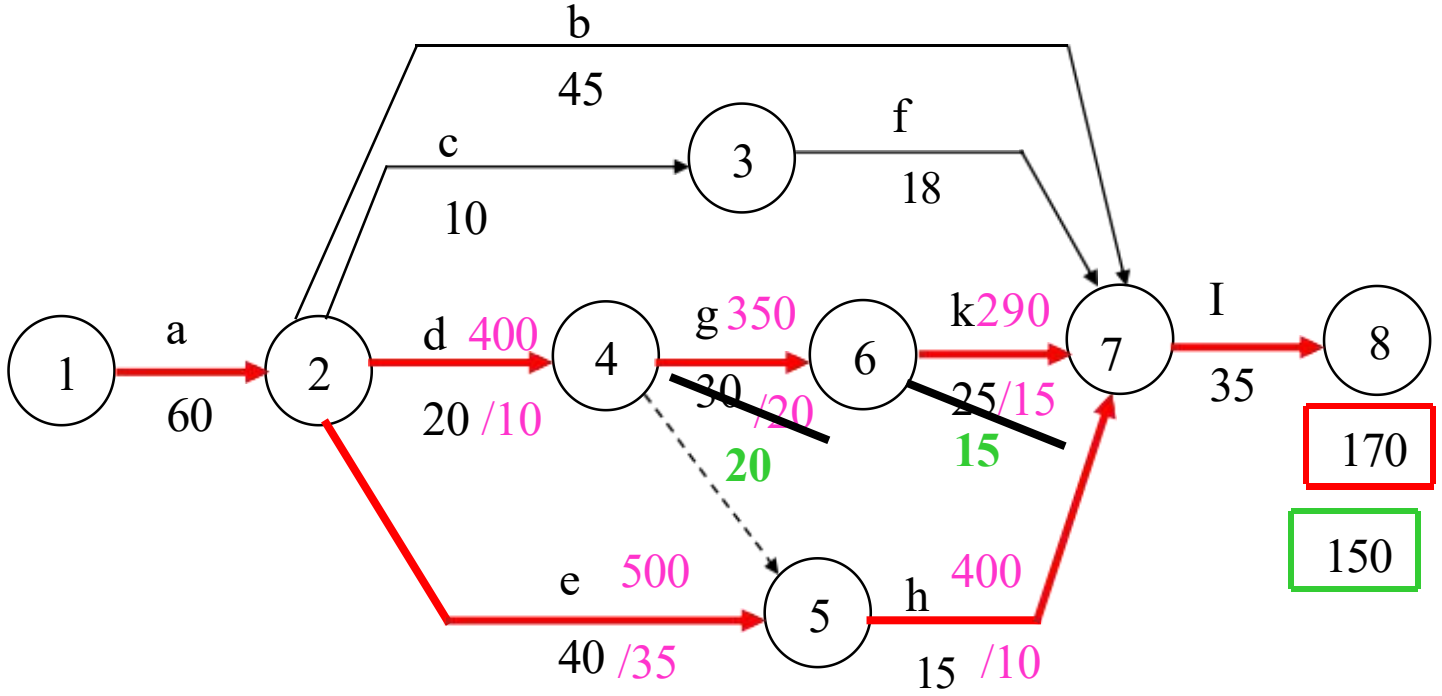


方案Ⅱ：*k,g*分别压缩10天，则总工期为150天，在方案Ⅰ的基础上：

节省$20\times400-(290\times10+350\times10)=1600$（元）

总费用 = 136900 - 1600 = 135300（元）

还可以继续尝试压缩其他关键路径上的工序时间。



方案Ⅲ：由于有两条平行的关键路线，只有同时压缩时间才能使得总的完工期缩短，而*g*,*k*工序已经压缩到了极限，则在方案Ⅱ的基础上同时压缩*d*、*e*或者*d*、*h*:

d+*e*节省: 5×400 - (400 + 500)×5=-2500(元)

d+*h*节省: 5×400 - (400 + 400)×5=-2000(元)

} 不利

最优方案为Ⅱ：工程完工期150天，总费用135300元

- 超前(Lead)与滞后(Lag)
- 时间压缩法
- 资源优化

资源优化

- 资源平衡
- 资源平滑

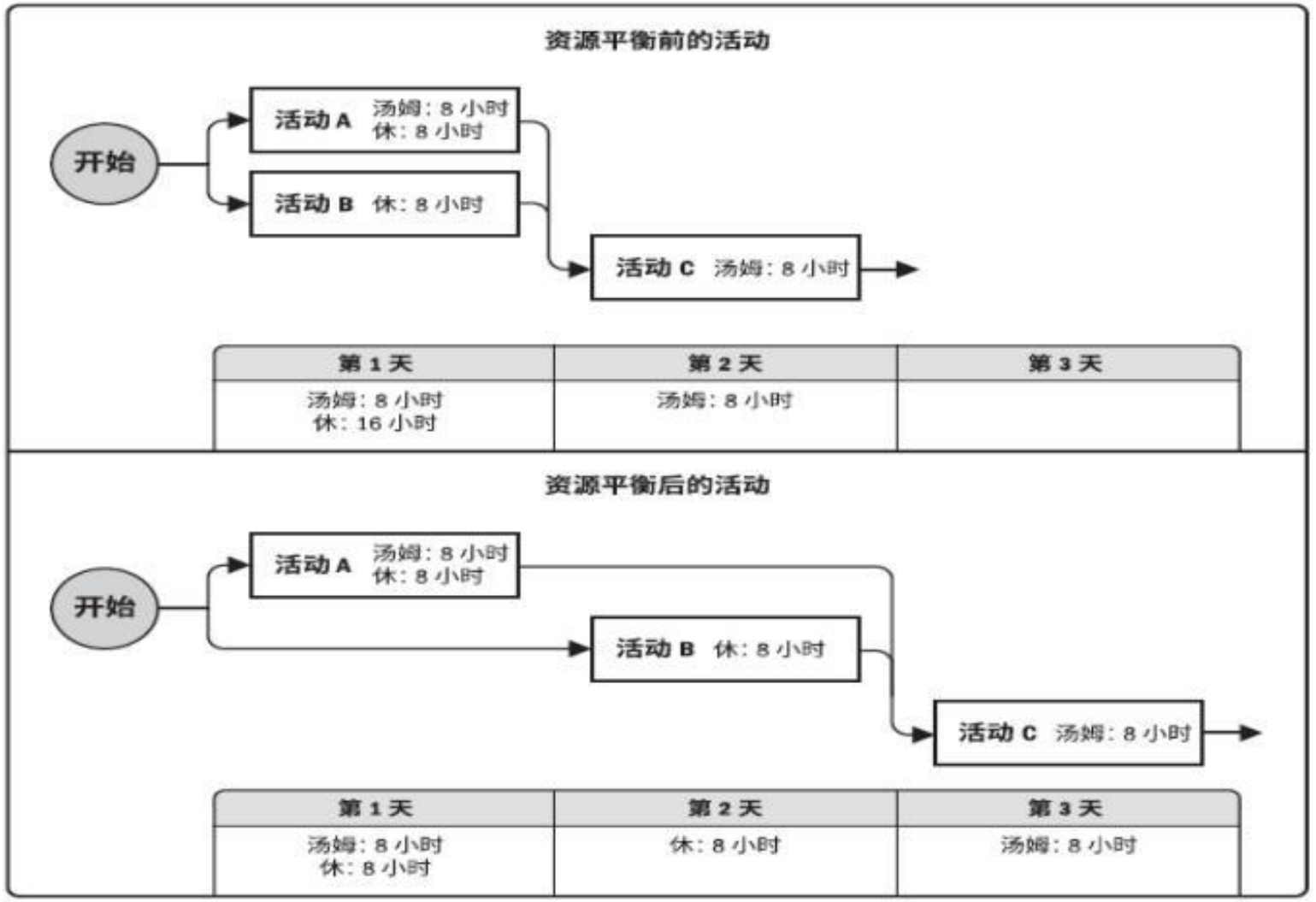
- **资源平衡法**

✓ 为了在资源需求与资源供给之间取得平衡，根据资源制约因素对开始日期和完成日期进行调整的一种技术。

✓ 通过调整任务的时间来协调资源的冲突。

✓ 资源平衡往往导致关键路径改变。

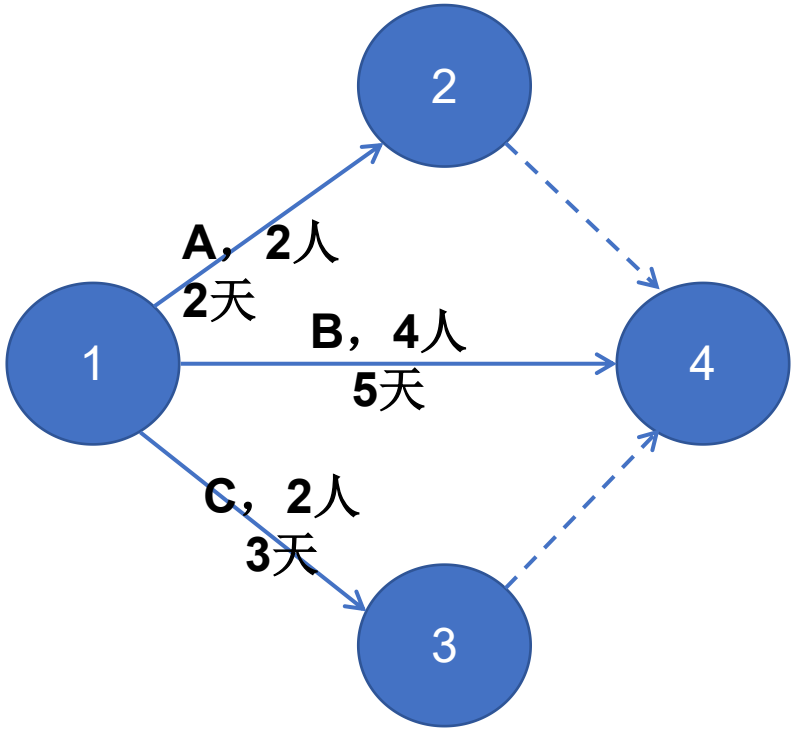
- **资源平衡法**


资源平衡前的活动
开始
活动 A 汤姆：8 小时 休：8 小时
活动 B 休：8 小时
活动 C 汤姆：8 小时
第 1 天
第 2 天
第 3 天
汤姆：8 小时 休：16 小时
汤姆：8 小时
资源平衡后的活动
开始
活动 A 汤姆：8 小时 休：8 小时
活动 B 休：8 小时
活动 C 汤姆：8 小时
第 1 天
第 2 天
第 3 天
汤姆：8 小时 休：8 小时
休：8 小时
汤姆：8 小时

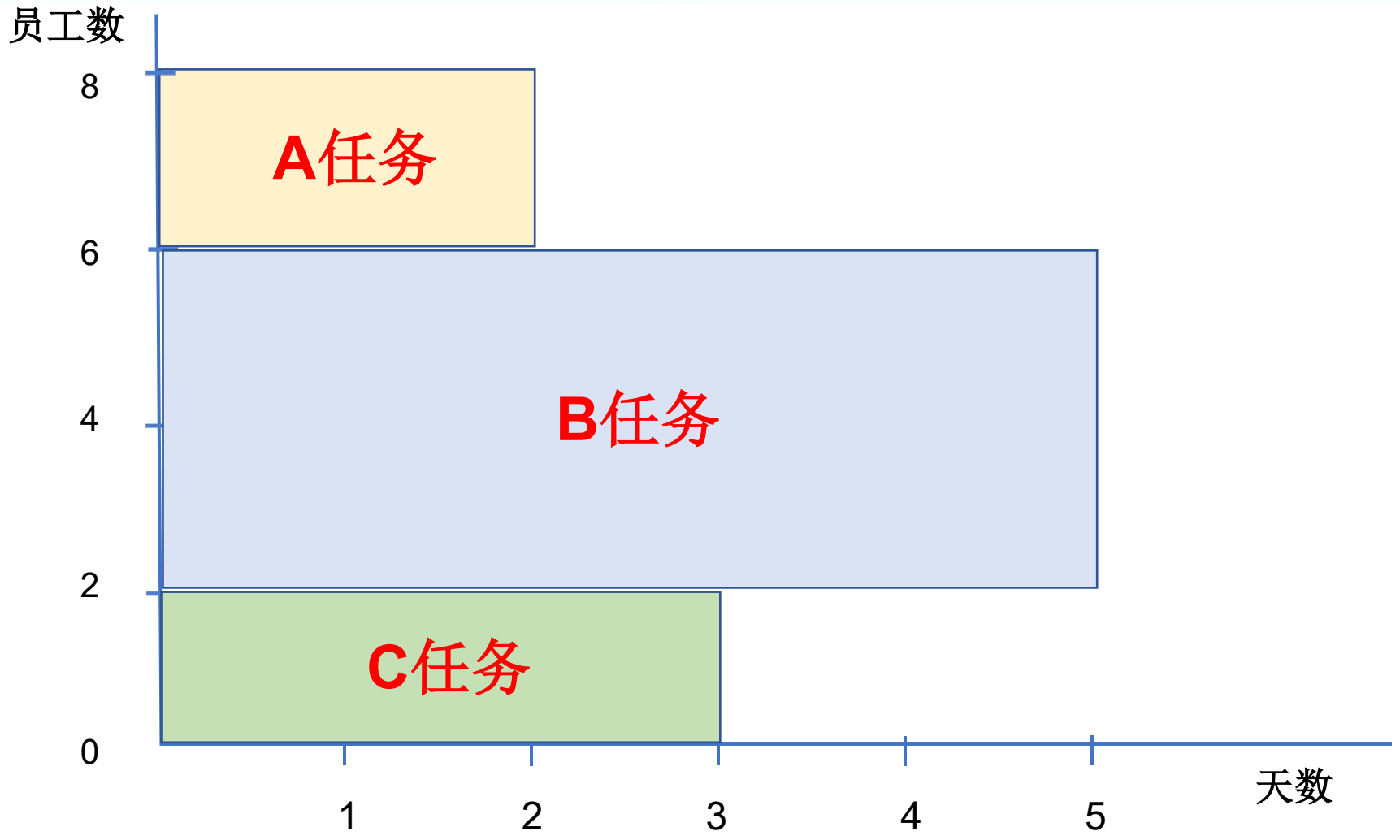
- **资源平滑法**

✓ 资源平滑法是在项目编排中进行资源的优化配置，保证资源最优化、最有效。

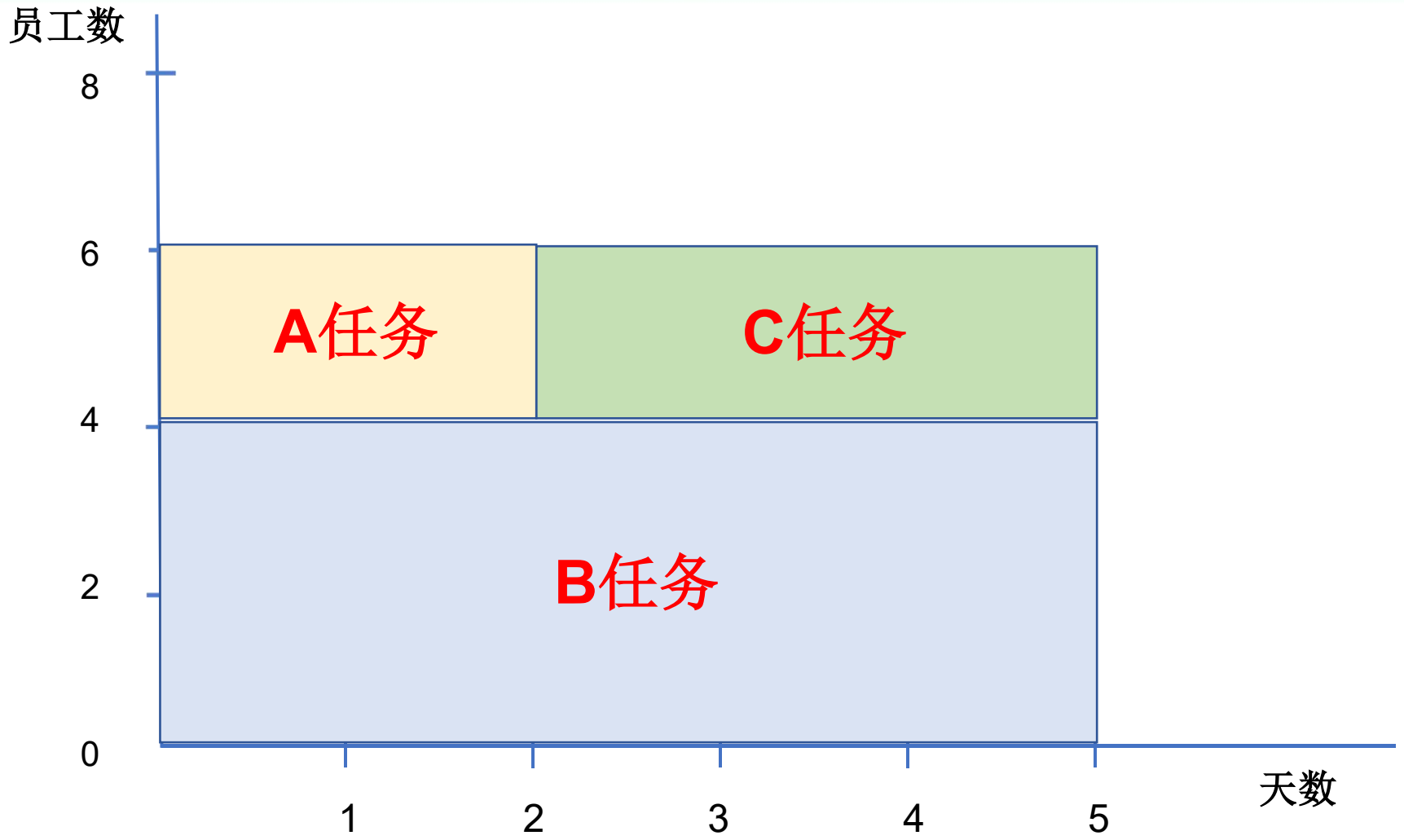
✓ 资源平滑不会改变项目关键路径，完工日期也不会延迟。活动只在其自由和总浮动时间内延迟。



某项目工程网络技术图如左，共三项任务，任务A需要2名员工、2天时间完成，任务B需要4名员工、5天时间完成，任务C需要2名员工、3天时间完成。

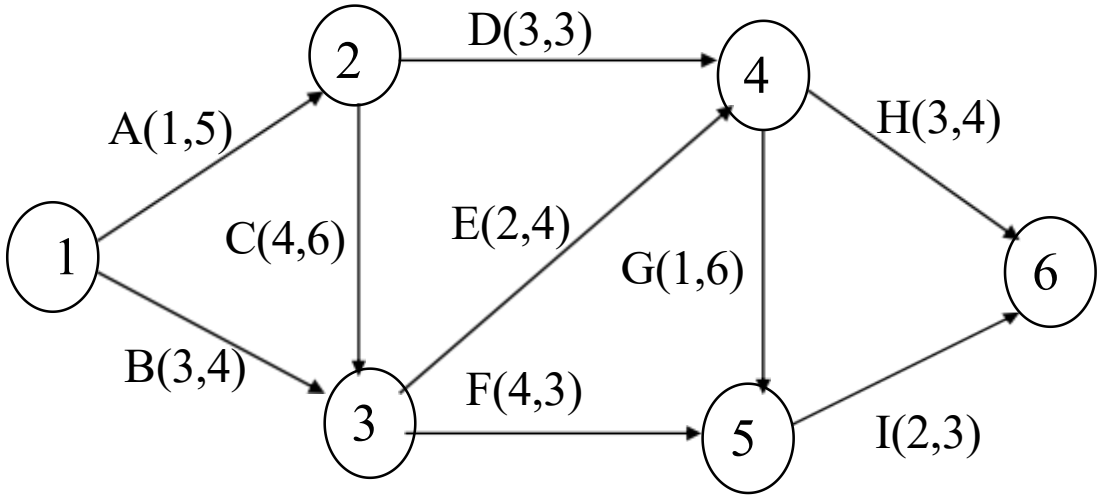


所有任务从第1天开始时甘特图



平滑后C任务晚开始2天

例：某工程网络计划如图，括号内表示时间、所需电力。若每天供电10单位，试合理安排各工序进度。



解：①计算总完工期、确定关键路线；

②计算各工序的总时差R(i,j)；

③作甘特图。

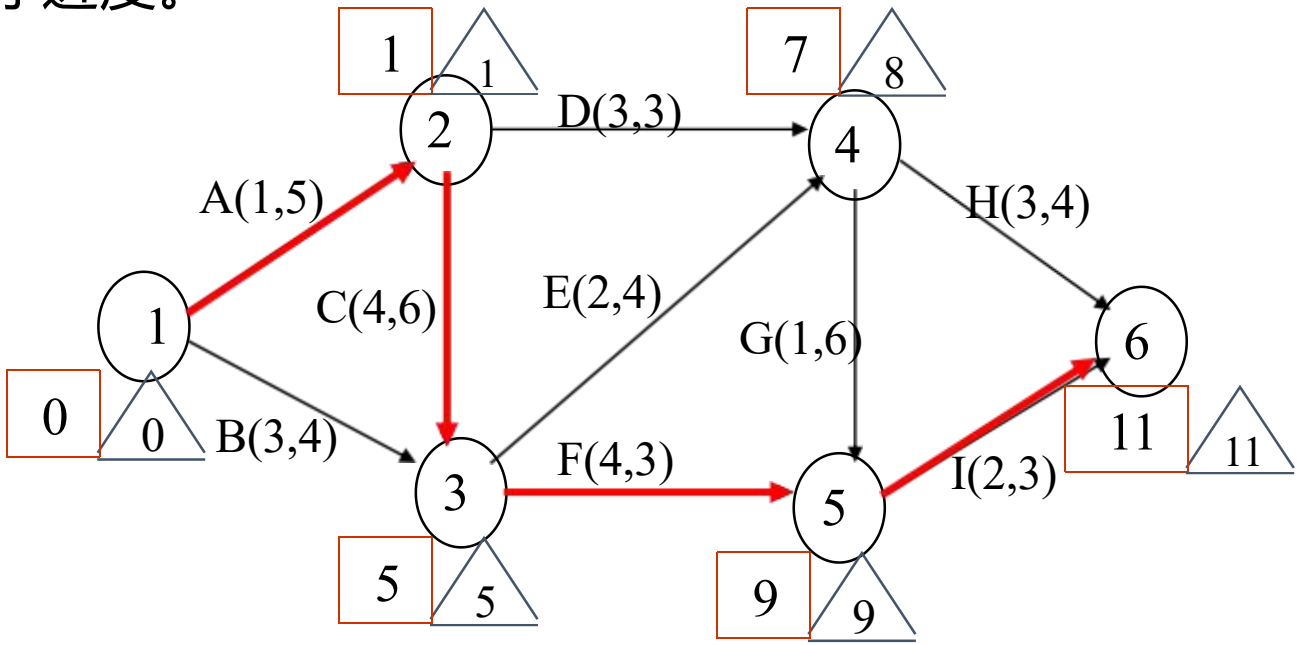
优化方法：

(1)优先安排关键工序所需要的资源；

(2)利用非关键工序的总时差，调整其开始时间，以错开资源需求高峰；

(3)调整进度安排，以满足资源限制，并使总完工期拖延最少。

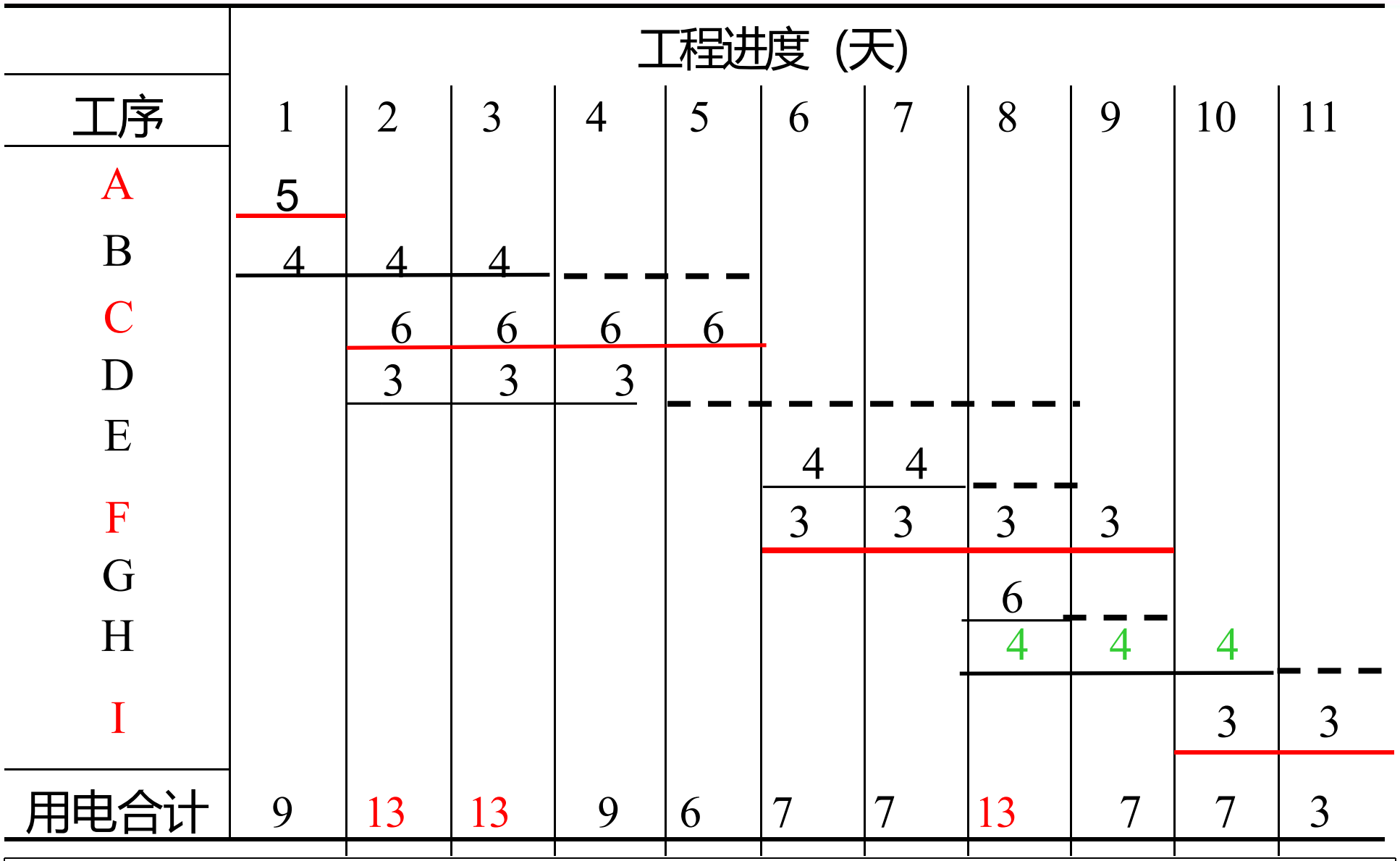
例：某工程网络计划如图，括号内为时间和所需电力。若每天供电10单位，试合理安排各工序进度。



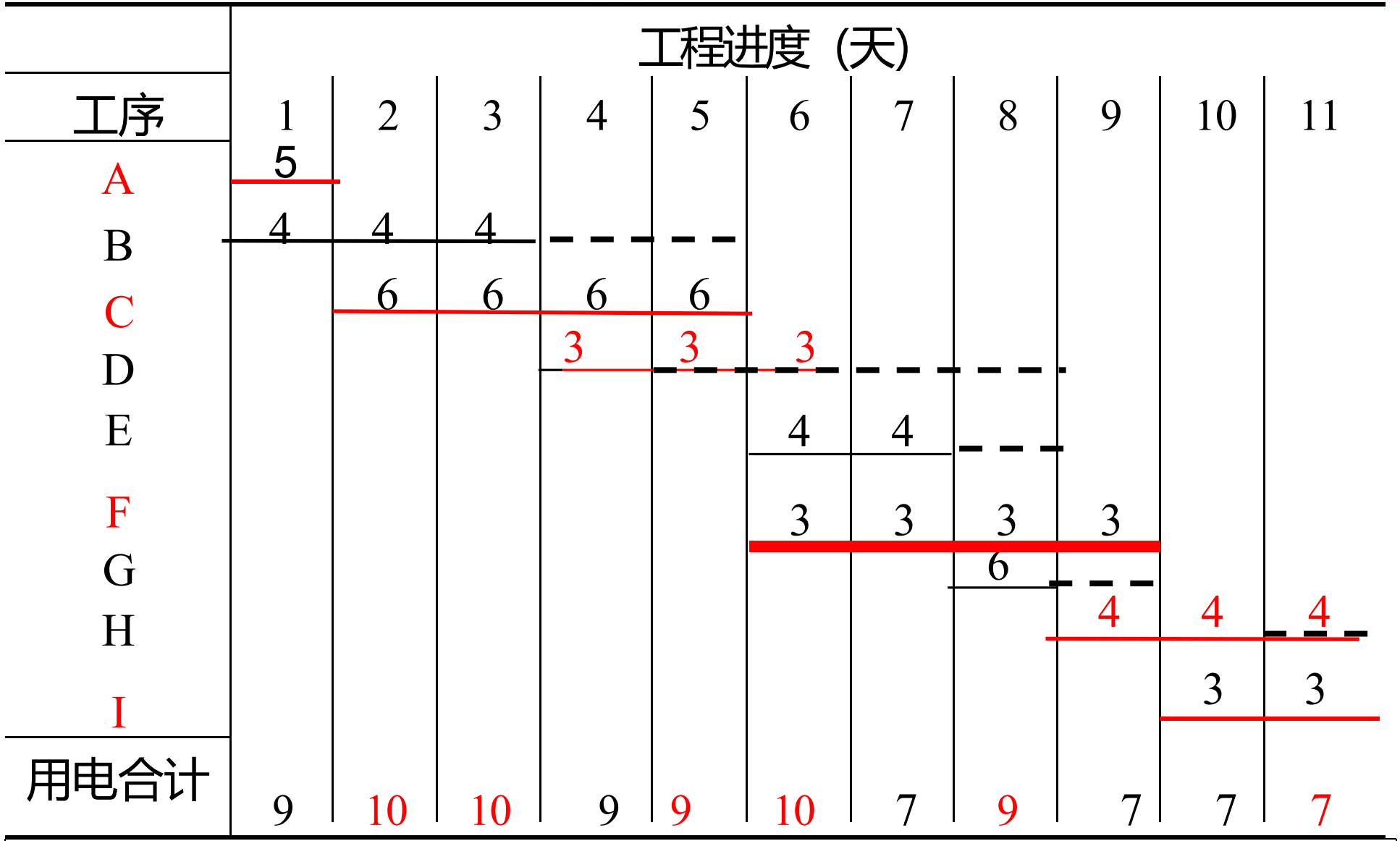
解：①计算总完工期、确定关键路线；

②计算各工序的总时差R(i,j)；

③作甘特图。



—— 关键工序； —— 非关键工序； - - - 非关键工序总时差。

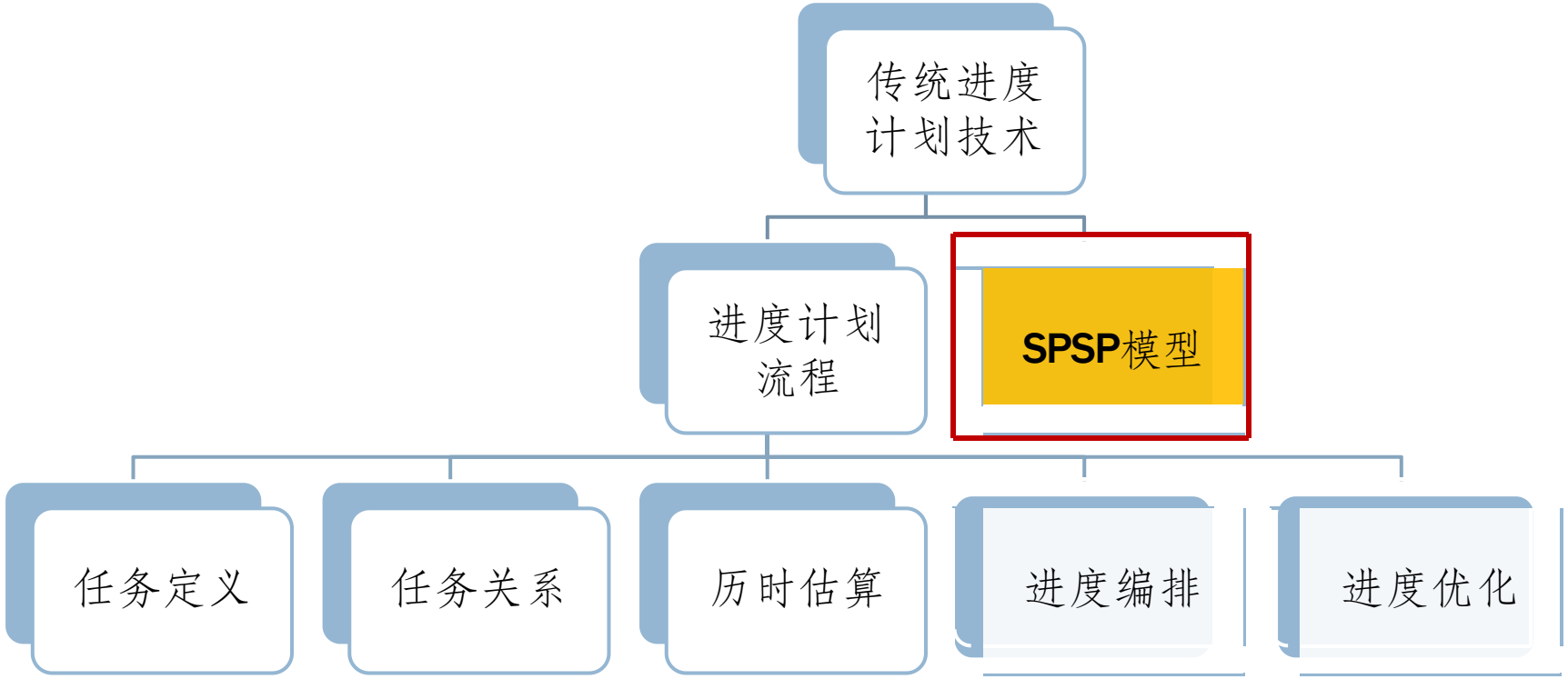

工程进度（天）
工序
1 2 3 4 5 6 7 8 9 10 11
A
B
C
D
E
F
G
H
I
用电合计
5
4 4 4
6 6 6 6
3 3 3
4 4
3 3 3 3
6
4 4 4
3 3
9 10 10 9 9 10 7 9 7 7 7
—— 关键工序； —— 非关键工序； - - - 非关键工序总时差。

计划优化调整

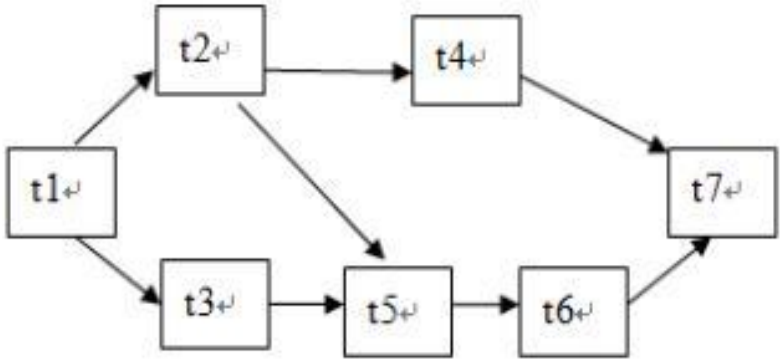
- 调整资源，解决资源冲突；
- 调整进度，优化项目，缩短工期；
- 调整项目成本预算，以便减少项目费用。



传统进度计划知识点



- **软件项目进度问题(Software Project Scheduling Problem,SPSP)模型：**在给定项目的任务、任务之间关系和资源限制下，对项目确定合适的人员安排，以保证项目的时间最短、成本最小。



+



1. 任务T={t_1,t_2,...,t_m}
   - 任务t_j所需要具备的技能t_j^{sk}；
   - 任务t_j的工作量t_j^{eff}；
2. 人员E={e_1,e_2,...,e_e},
   - e_i员工的技能集合e_i^{sk}；
   - e_i员工对项目的相对贡献度e_i^{maxd}，如果$e_i^{maxd} < 1$，则表示此人为兼职；
   - e_i^{rem}代表e_i员工的人力成本；
3. 技能S={s_1,s_2,...,s_n}：任务T所需要的所有技能集合。

已知任务T共包含7个子任务：$\{t_1,t_2,t_3,t_4,t_5,t_6,t_7\}$，所需要的技能S有3种：$\{s_1,s_2,s_3\}$，每一项子任务所需要的技能如下：

$$t_1^{sk}=\{s_1,s_2,s_3\},\ t_2^{sk}=\{s_1,s_2\},\ t_3^{sk}=\{s_2,s_3\},\ t_4^{sk}=\{s_2\},$$
$$t_5^{sk}=\{s_1,s_2,s_3\},\ t_6^{sk}=\{s_1,s_2\},\ t_7^{sk}=\{s_2\}$$

每一项子任务所需要的工作量如下：

$$t_1^{eff}=4,\ t_2^{eff}=6,\ t_3^{eff}=8,\ t_4^{eff}=6,$$
$$t_5^{eff}=8,\ t_6^{eff}=10,\ t_7^{eff}=16$$

围绕此项目有4名员工可用，$E=\{e_1,e_2,e_3,e_4\}$，每人所具备的技能以及每人的人力成本如下：

$$e_1^{sk}=\{s_1,s_2,s_3\}\ e_2^{sk}=\{s_1,s_2,s_3\},e_3^{sk}=\{s_1,s_2\}\ e_4^{sk}=\{s_2,s_3\}$$

$$e_1^{rem}=\$6000,e_2^{rem}=\$6000,e_3^{rem}=\$5000,e_4^{rem}=\$5000$$

- **员工对任务的付出矩阵**

$$\begin{bmatrix} m_{11} & m_{12} & \cdots & m_{1n} \\ m_{21} & m_{22} & \cdots & m_{2n} \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ m_{m1} & m_{m2} & \cdots & m_{mn} \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0.5 & 0.5 & 0 & 1 & 1 & 1 \\ 1 & 0.5 & 0.5 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0.5 & 0 & 0.5 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$m_{ij}<=1$：第i个人对第j个任务的付出

任务历时计算

$$t_j^{len} = \frac{t_j^{eff}}{\sum_{i=1}^{|e|} m_{ij}}$$

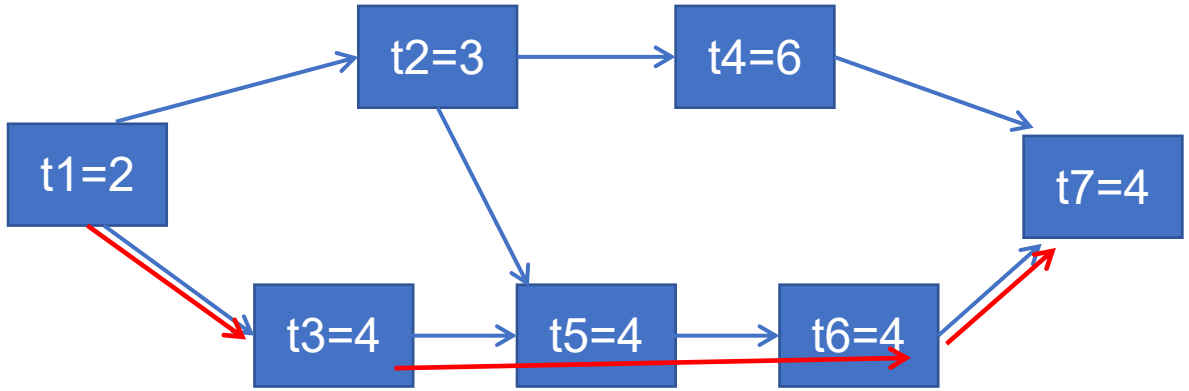
$$t_1^{len} = \frac{4}{1+1+0+0} = 2$$

$t_1^{eff} = 4, t_2^{eff} = 6, t_3^{eff} = 8, t_4^{eff} = 6,$

$t_5^{eff} = 8, t_6^{eff} = 10, t_7^{eff} = 16$

$$\left|\begin{array}{ccccccc} 1 & 0.5 & 0.5 & 0 & 1 & 1 & 1 \\ 1 & 0.5 & 0.5 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0.5 & 0 & 0.5 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 \end{array}\right|$$

任务历时计算



关键路径：t1-t3-t5-t6-t7，路径长度：18

$\{t_1, t_2, t_3, t_4, t_5, t_6, t_7\}$ = {“Planning”, “Low-level design”, “High-level design”, “Testcase design”,“Coding”, “System Testing ”, “Acceptance Testing ”, }

成本计算

Cost1=2×(6000+6000)=24,000

Cost2=3×(0.5×6000+0.5×6000+5000) =33,000

Cost3=4×(0.5×6000+0.5×6000+5000) =44,000

Cost4=6×(0.5×5000+5000) =45,000

Cost5=4×(6000+6000) =48,000

Cost6=4×(6000+6000+0.5×5000) =58,000

Cost7=4×(6000+6000+5000+5000) =44,000

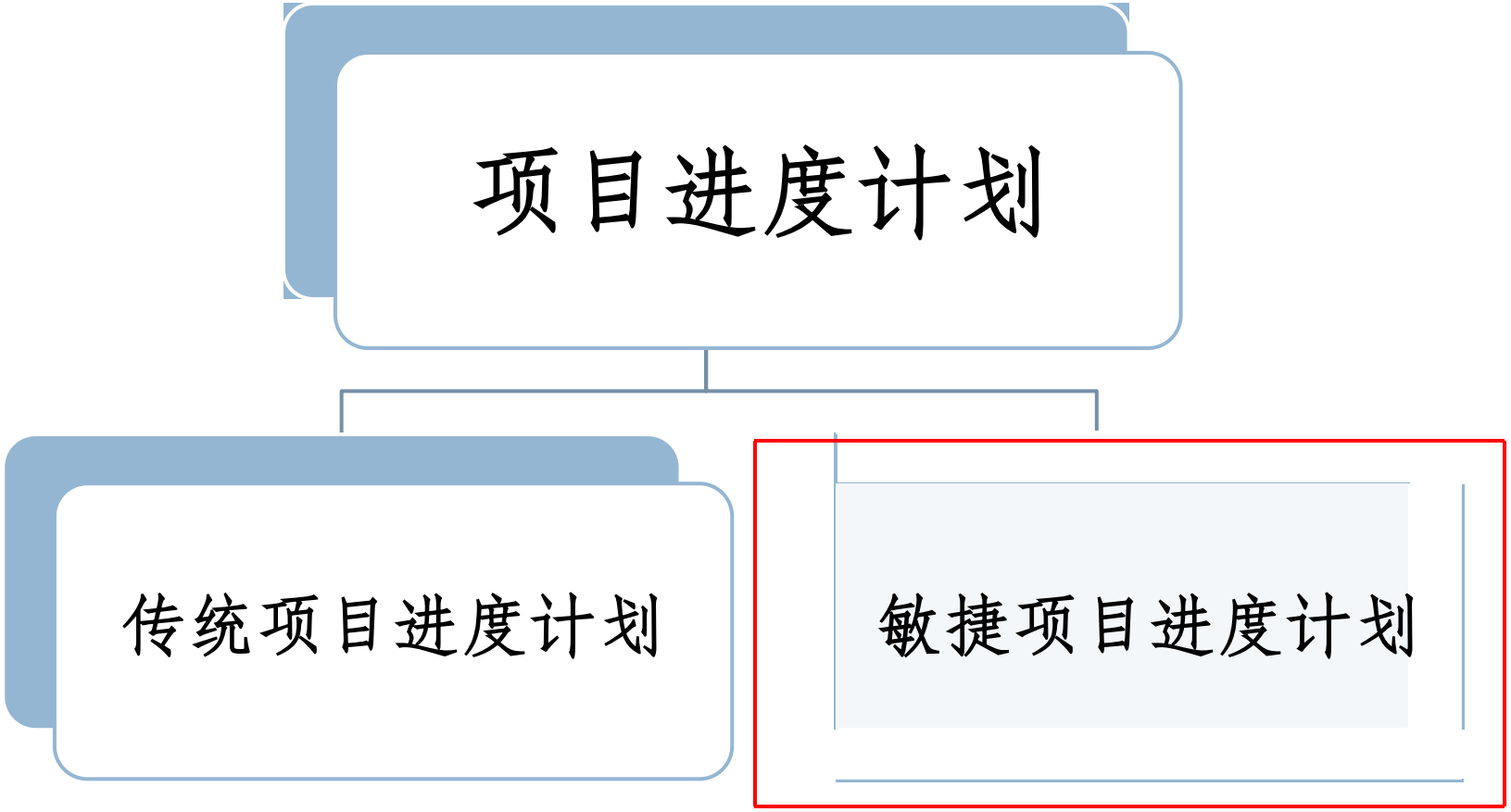
总成本Cost=340,000

- 项目进度编排的目标

目标函数：经济模型

$$f(x) = w^{cos} p^{cos} + w^{len} p^{len}$$

这是W^{cos}指是平均成本的倒数，P^{cos}指的是计算得出的成本；W^{len}指的是平均进度的倒数，P^{len}指的是计算得出的进度。目标结果则是求f(x)达到最小。

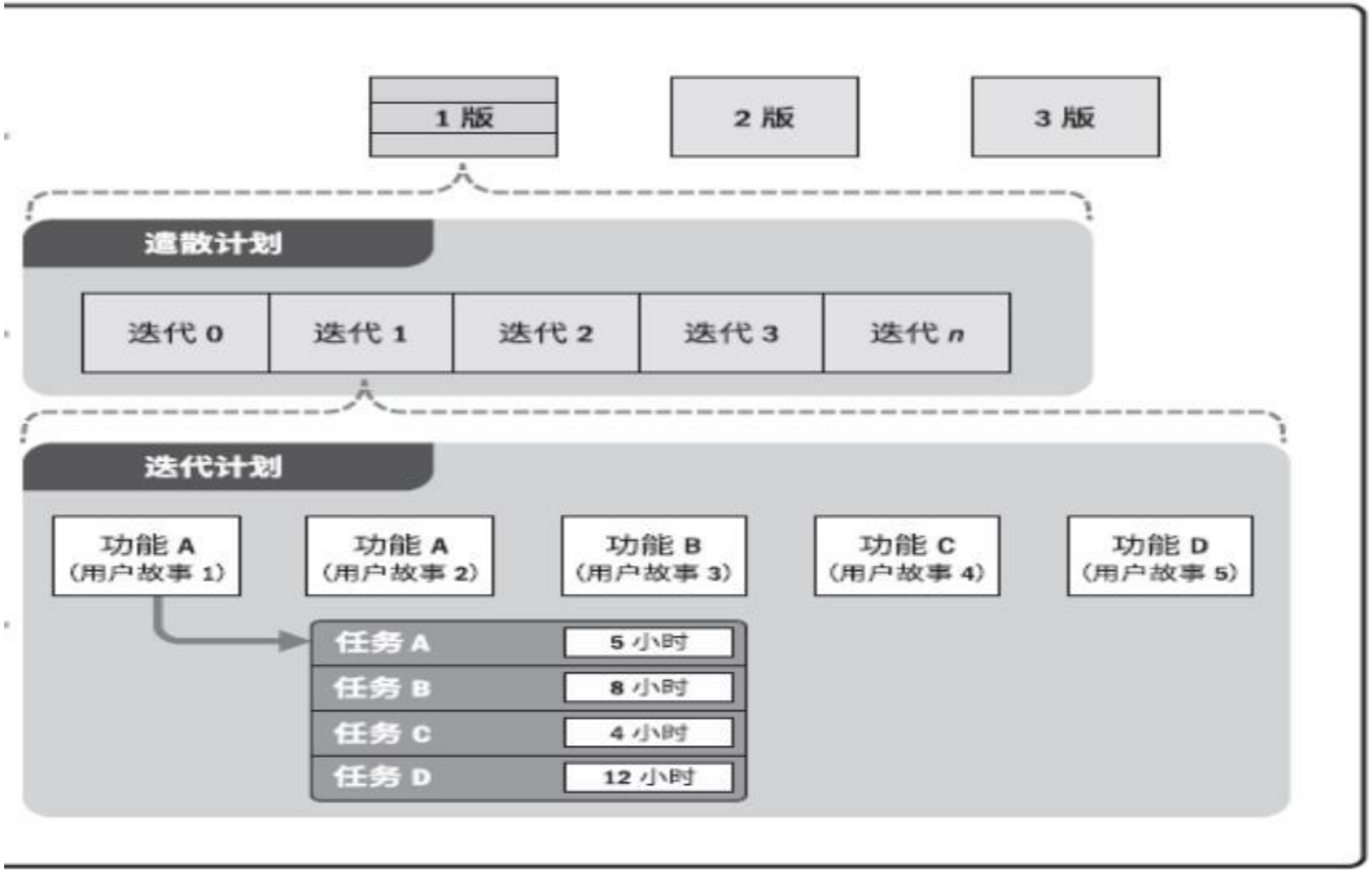

项目进度计划
传统项目进度计划
敏捷项目进度计划

- **敏捷计划编排（Agile Planning）**

✓ **Release planning** - 发布计划
  - 远期计划- 粗计划.

✓ **Iteration planning** - 迭代计划
  - 近期计划- 细计划

发布计划与迭代计划的关系



- **迭代计划**

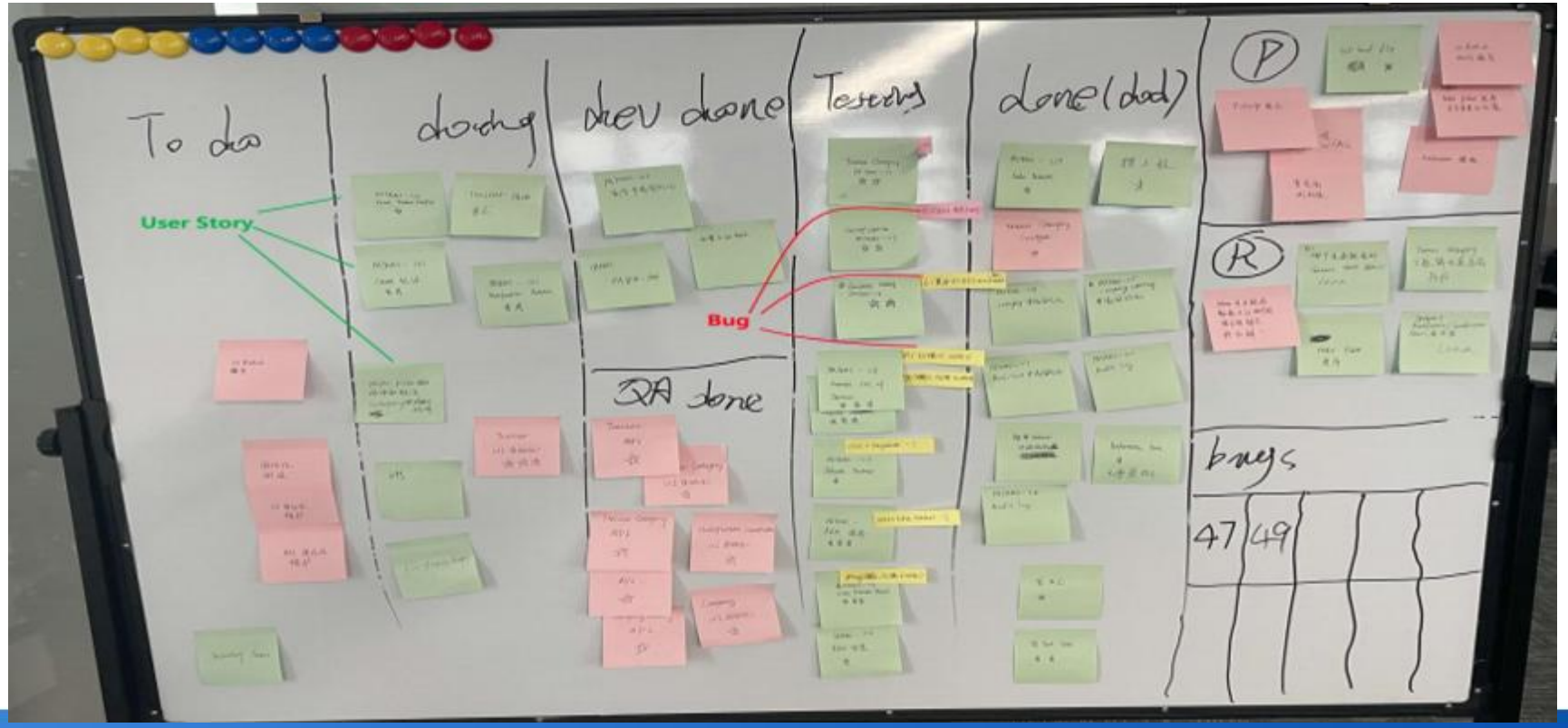
✓ 确定迭代周期-时间盒（TimeBox）
✓ 培养团队的工作节奏
✓ 确定关键工作任务
✓ 周期内完成所有任务，不要拉长迭代时间

迭代0：开发准备工作

- ✓ 需求分析（Backlog、Story Map）
- ✓ 架构设计（例如B/S、C/S架构模式）
- ✓ 开发环境
- ✓ 确定时间盒
- ✓ 系统原型（如静态图示）
- ✓ ……

迭代i:

- ✓ TimeBox：1-4周
- ✓ 价值交付
- ✓ 看板: ToDo-Doing-Done-Testing-Finish



- **历时估算的基本方法**

1. 开发速度稳定前
2. 开发速度稳定后

- 开发速度**Scrum Velocity**：是指在Scrum框架中，一个开发团队在特定冲刺期间完成的工作量的度量，是Scrum中的关键指标。例如，在冲刺结束时完成的故事点数。

1.开发速度稳定前，估计开发速度

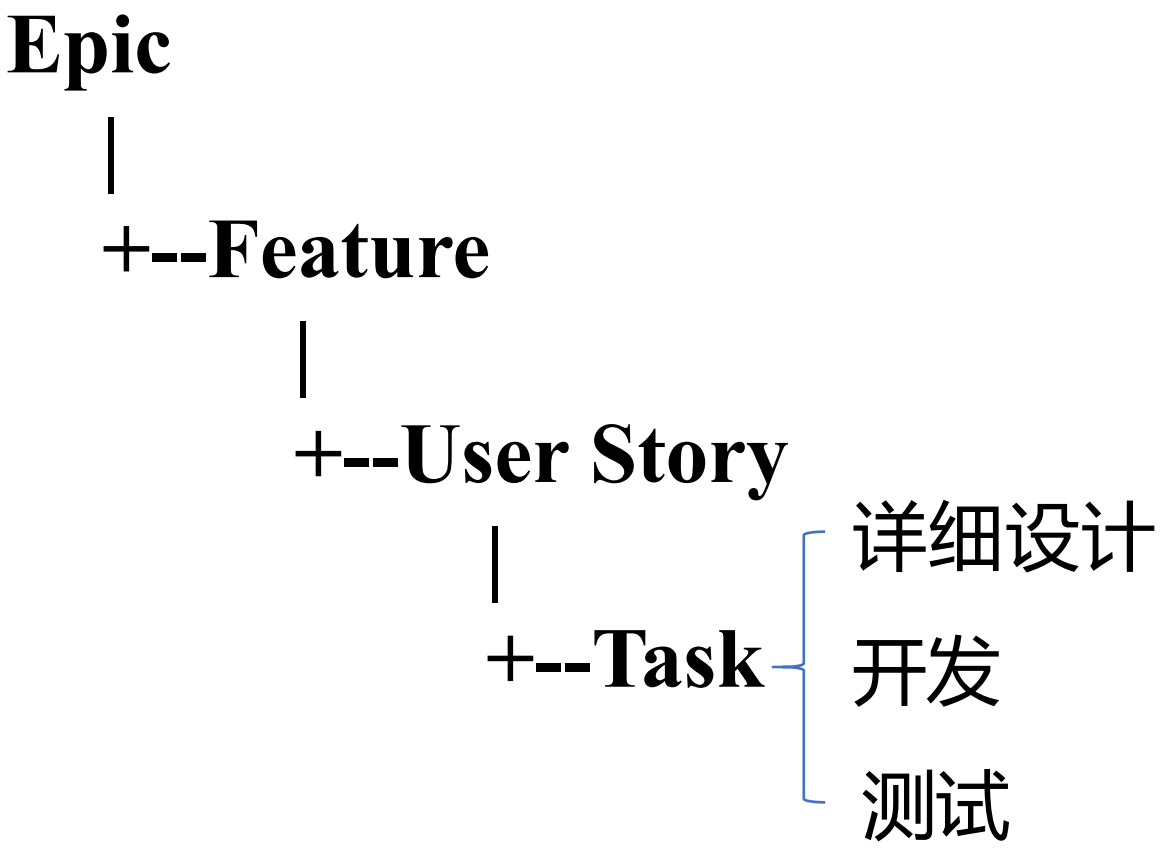
- 快速故事点方法(**A Fast Story Point Estimation**)
- 扑克游戏估算 (Planning Poker)

2.开发速度稳定后，估计开发速度

- 基于迭代生产率的估算
- 基于故事点生产率的估算

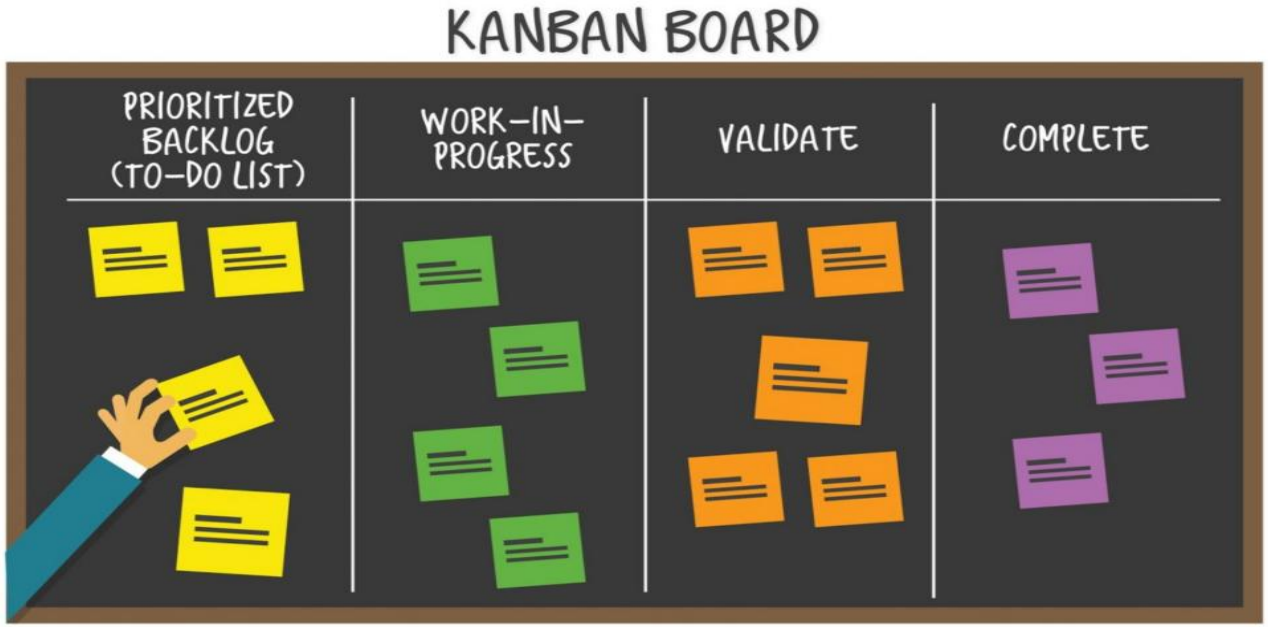
- **基于迭代生产率的历时估算：**如果**Scrum Velocity** =10个Story points/迭代，还有100 Story points，那么，项目大约还需要的历时：10个迭代。
- **基于Story Point生产率的历时估算：**如果平均生产率为3天/Story point，项目共计30 Story points。那么，项目历时 = 90工作日(大约4-5个月)。

- **敏捷任务分解，Story--->Task**



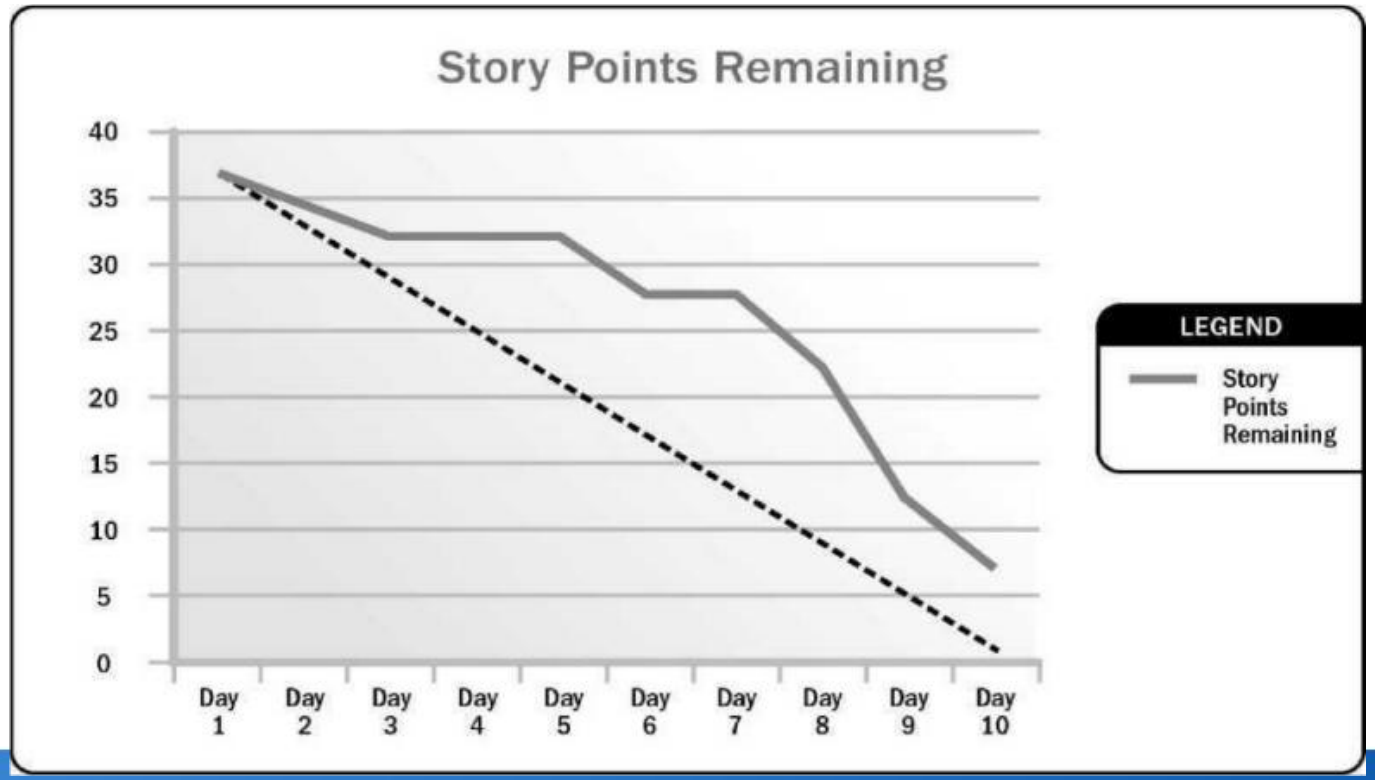
进度管理图示

- 看板（Kanban）
- 燃尽图(Burndown Chart)
- 燃起图(Burnup Chart)



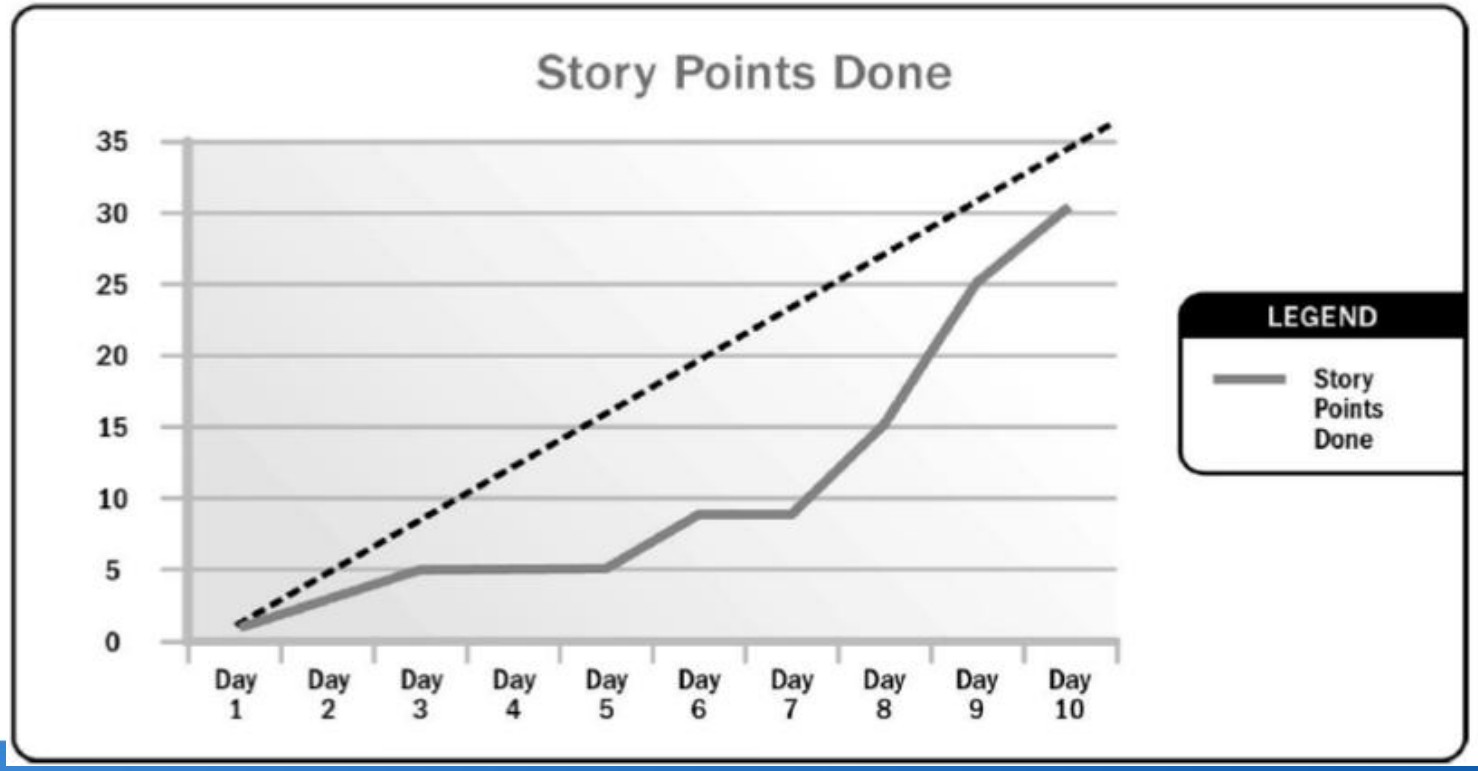
进度管理图示

- 看板（Kanban）
- **燃尽图(Burndown Chart)**
- 燃起图(Burnup Chart)

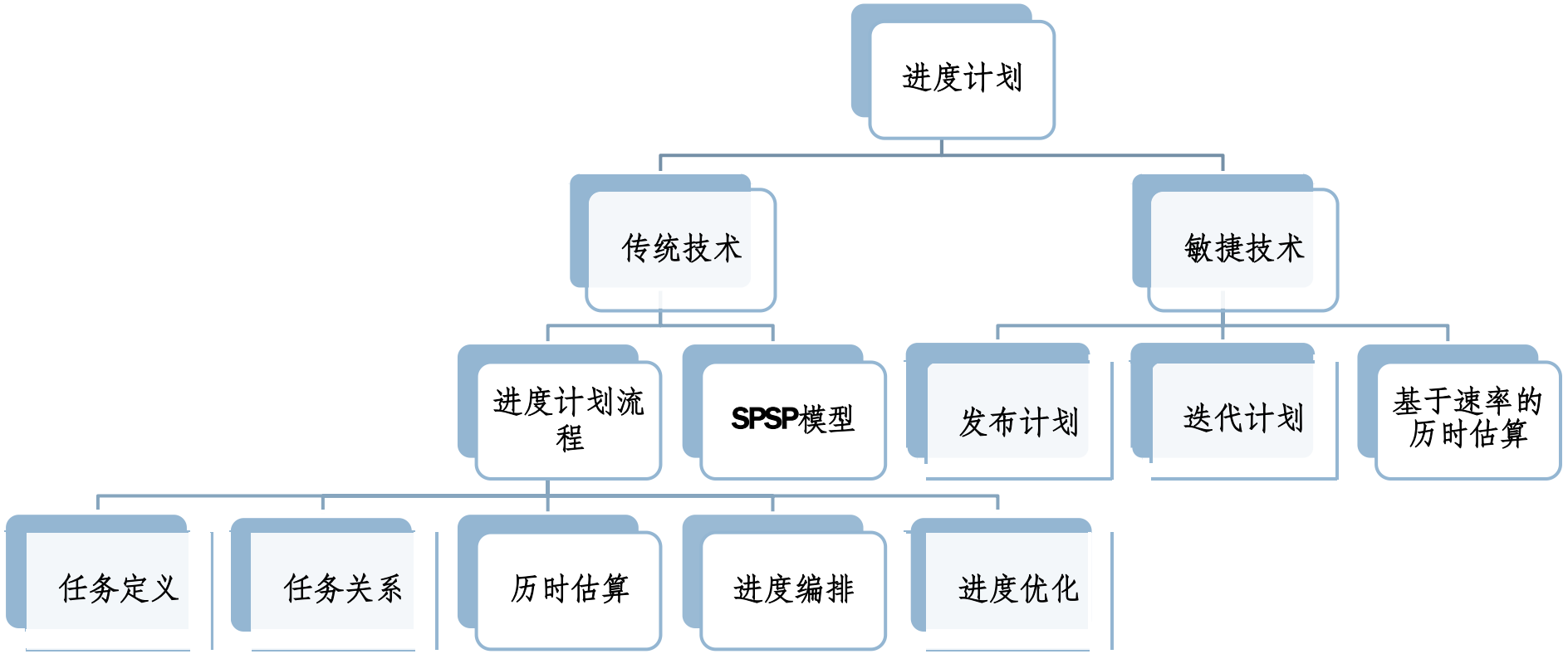


进度管理图示

- 看板（Kanban）
- 燃尽图 (Burndown Chart)
- **燃起图 (Burnup Chart)**



本章小结－核心之“道”


进度计划
传统技术
敏捷技术
进度计划流程
SPSP模型
发布计划
迭代计划
基于速率的历时估算
任务定义
任务关系
历时估算
进度编排
进度优化